https://mithril-th.org/

Threshold MLDSA

∴

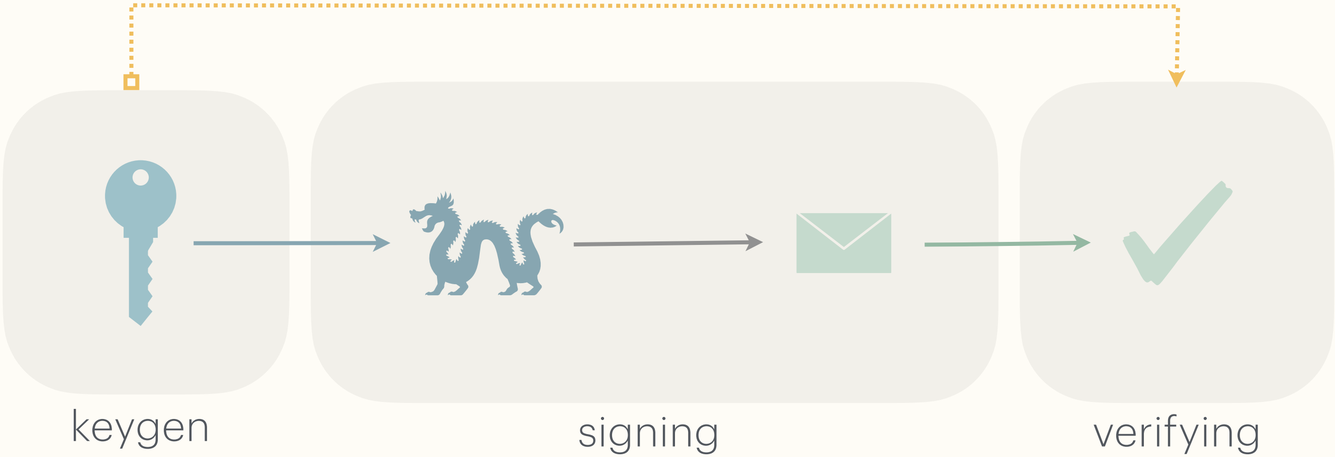
Mithril

A practical, *ML-DSA compatible* Threshold Signature Scheme

Sofia Celi | Brave & University of Bristol
Thomas Espitau | PQShield

What is a signature ?

2 party protocol to verify the **authenticity** of a message



What is a threshold signature ?

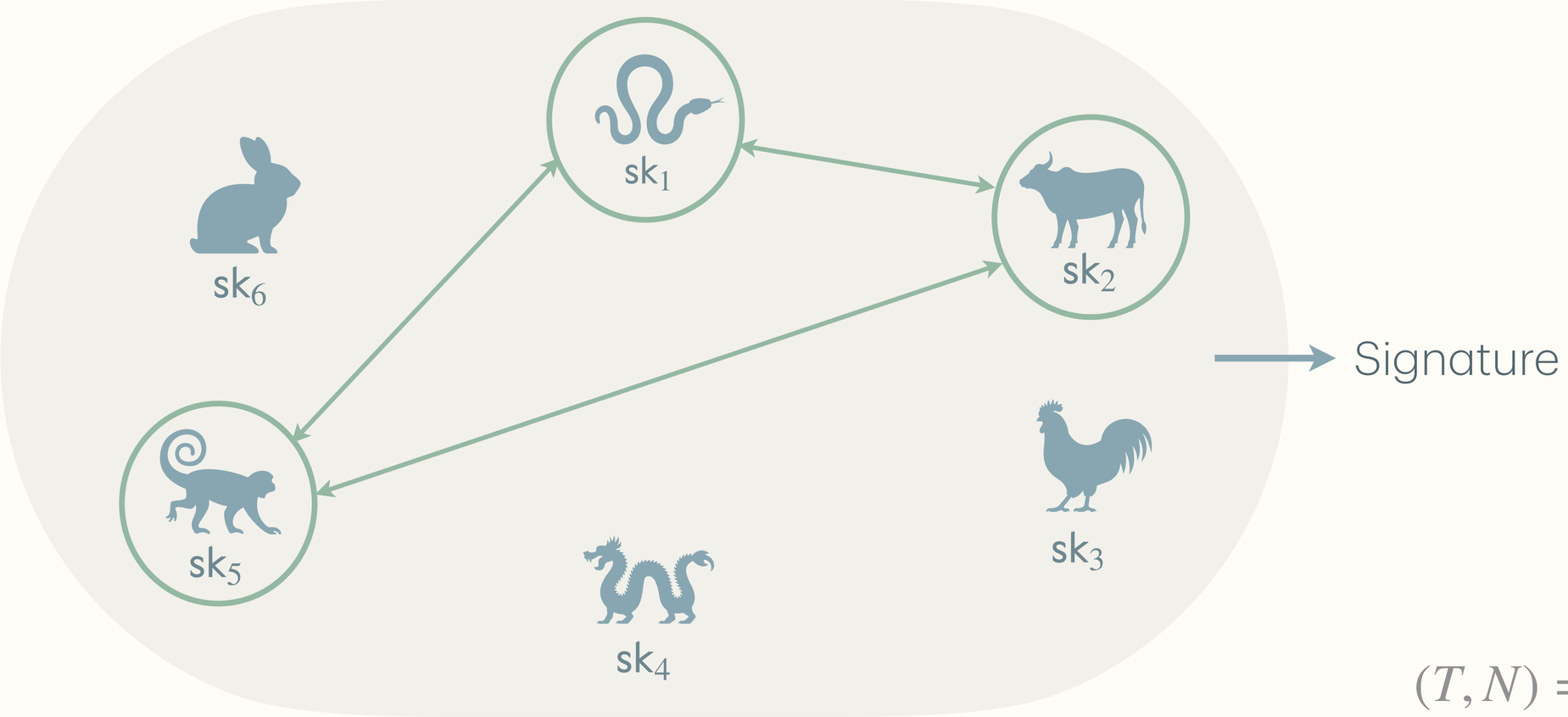
Interactive protocol to distribute signature generation such that:

T out of N parties can **collaborate** to sign a message and $T-1$ parties **cannot** sign.

What is a threshold signature ?

Interactive protocol to distribute signature generation such that:

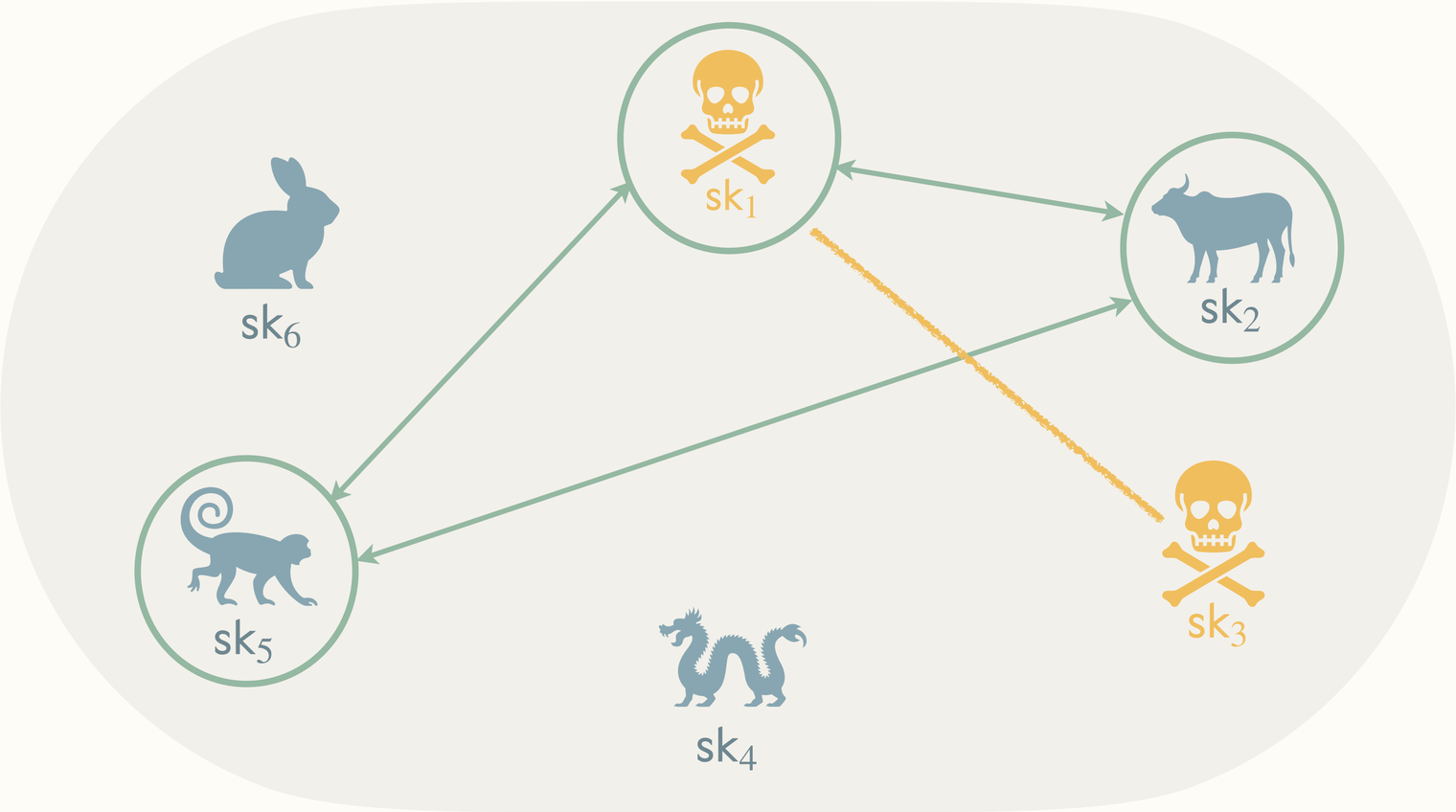
T out of N parties can **collaborate** to sign a message and $T-1$ parties **cannot** sign.

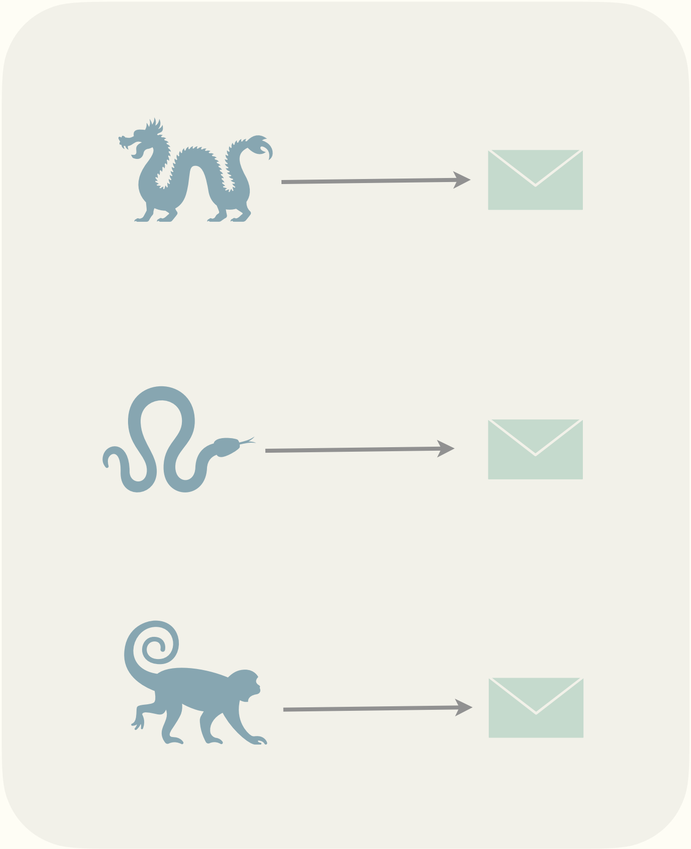


$(T, N) = (3,6)$

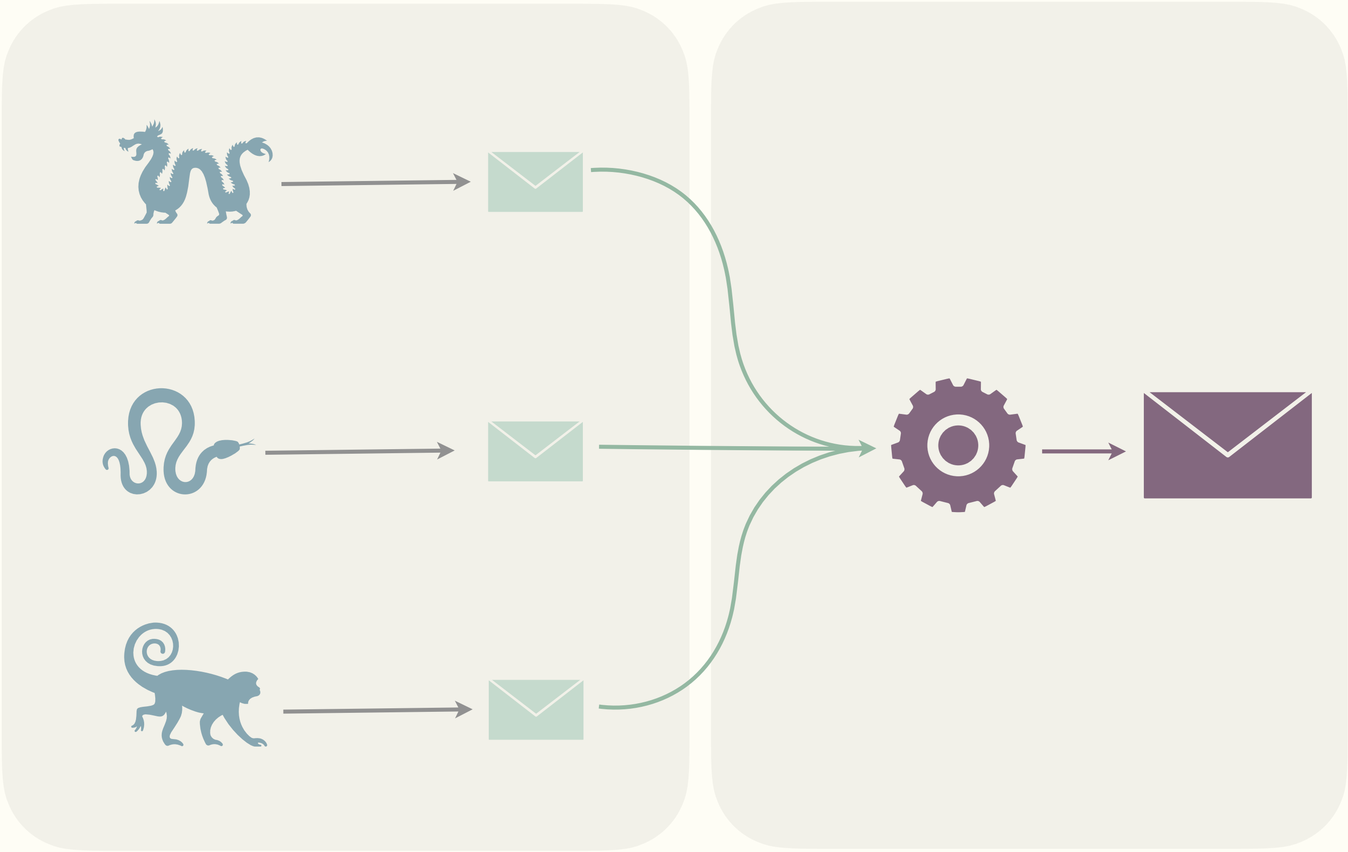
Security requirements

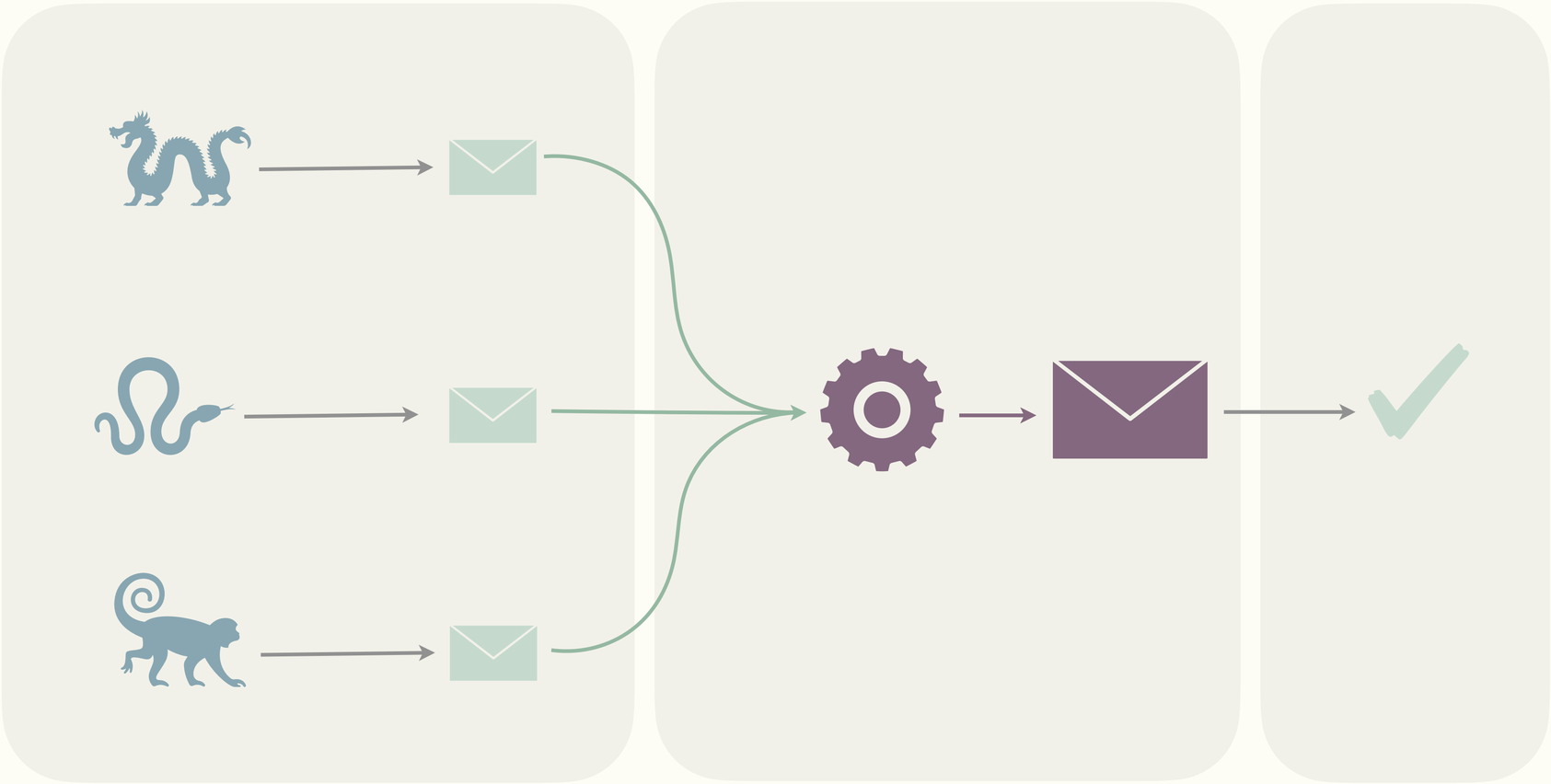
- **Correctness**: with at least T-out-of-N partial signing keys, we can sign.
- **Unforgeability**: remains unforgeable even if up to T' parties are corrupted, where $T' \leq T - 1$.

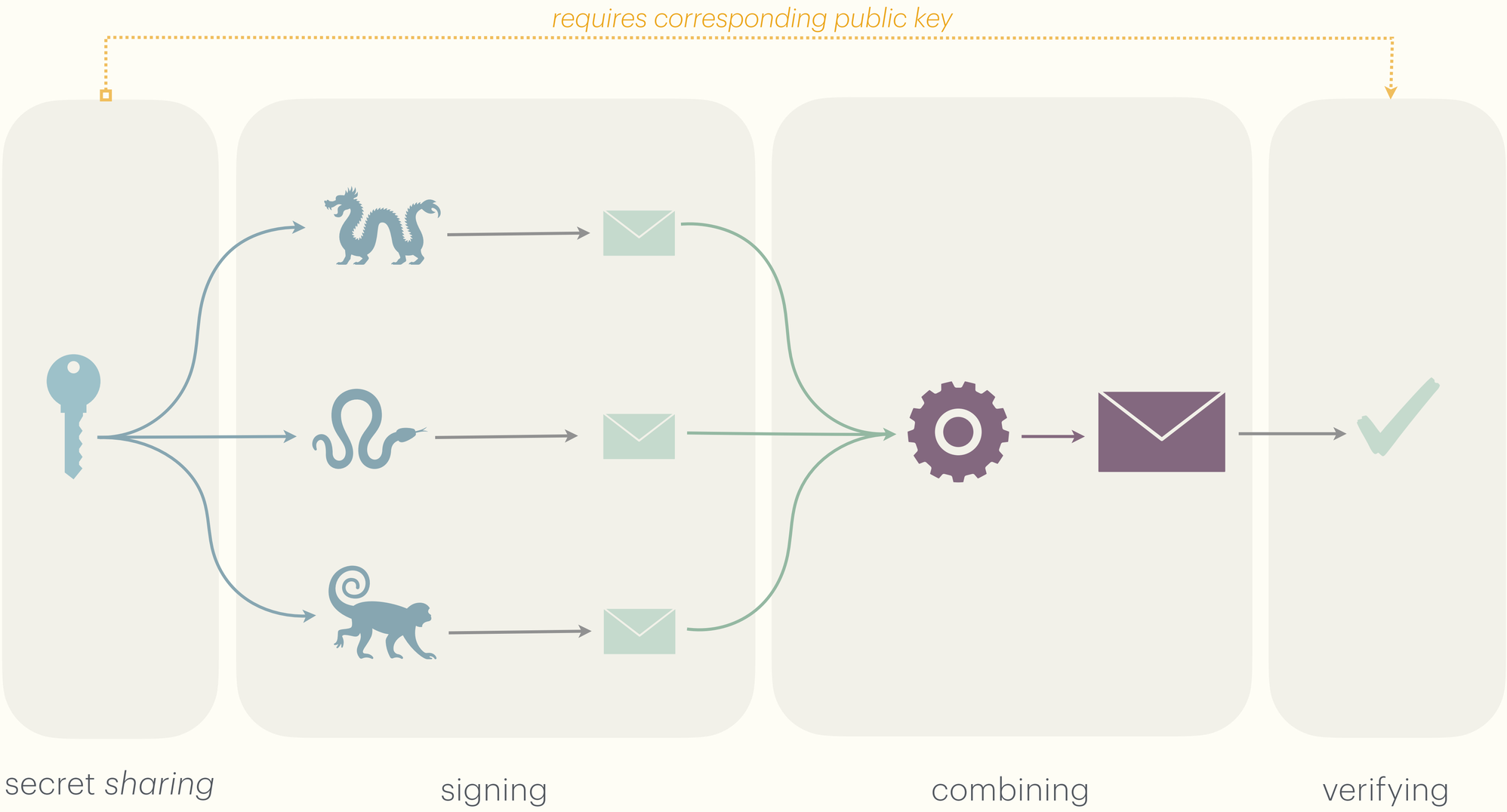




signing


signing
combining


signing
combining
verifying


requires corresponding public key
secret sharing
signing
combining
verifying

SIGNATURE SCHEME

+

KEY DISTRIBUTION / SHARING

=

THRESHOLD SIGNATURE

What can we do with lattices ?

- Create random *short* samples
- Hide with a random matrix
- Do linear algebra

Short integer solution (SIS)

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk} \pmod{q}$, for short sk

LWE

- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r} \pmod{q}$ for short sk , $\mathbf{r}$

(*flooded*) Fiat-Shamir 101

Simple identification protocol

Prover

Verifier

Simple identification protocol

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Prover

Verifier

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- vk $= \mathbf{A} \cdot$ sk, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

Verifier

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- vk $= \mathbf{A} \cdot$ sk, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

$\mathbf{w}$ →

Challenge

- Sample a short $\mathbf{c}$

Verifier

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

$\xrightarrow{\mathbf{w}}$

Challenge

- Sample a short $\mathbf{c}$

$\xleftarrow{\mathbf{c}}$

Answer

- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r}$

Verifier

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

$\xrightarrow{\mathbf{w}}$

Verifier

Challenge

- Sample a short $\mathbf{c}$

$\xleftarrow{\mathbf{c}}$

Answer

- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r}$

$\xrightarrow{\mathbf{z}}$

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

$\xrightarrow{\mathbf{w}}$

Verifier

Challenge

- Sample a short $\mathbf{c}$

$\xleftarrow{\mathbf{c}}$

Prover

Answer

- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r}$

$\xrightarrow{\mathbf{z}}$

Verifier

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot \mathsf{vk}$
- Assert $\mathbf{w}^* = \mathbf{w}$
- Assert $\mathbf{z}$ short

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- vk $= \mathbf{A} \cdot$ sk, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

Challenge

- Sample a short $\mathbf{c}$

Answer

- $\mathbf{z} = \mathbf{c} \cdot$ sk $+ \mathbf{r}$

$\mathbf{z}$ →

Verifier

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot$ vk
- Assert $\mathbf{w}^* = \mathbf{w}$
- Assert $\mathbf{z}$ short

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- vk $= \mathbf{A} \cdot$ sk, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

Challenge

- $\mathbf{c} =$ Hash$(\mathbf{w})$

Answer

- $\mathbf{z} = \mathbf{c} \cdot$ sk $+ \mathbf{r}$

$\mathbf{c}, \mathbf{z}$

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot$ vk
- Assert $\mathbf{w}^* = \mathbf{w}$
- Assert $\mathbf{z}$ short

Verifier

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$

Challenge

- $\mathbf{c} = \mathsf{Hash}(\mathbf{w})$

Answer

- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r}$

$\mathbf{c}, \mathbf{z}$ →

Verifier

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(\mathbf{w}^*)$
- Assert $\mathbf{z}$ short

(all operations are mod q)

Simple identification protocol

Keygen() → sk, vk

- vk $= \mathbf{A} \cdot$ sk, for short sk

Prover

Commit

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$
- $\mathbf{c} =$ Hash$(\mathbf{w})$
- $\mathbf{z} = \mathbf{c} \cdot$ sk $+ \mathbf{r}$

$\mathbf{c}, \mathbf{z}$ →

Verifier

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot$ vk
- Assert $\mathbf{c} =$ Hash$(\mathbf{w}^*)$
- Assert $\mathbf{z}$ short

(all operations are mod q)

Fiat-Shamir on lattices

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Sign

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$
- $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w})$
- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r}$
- Output $\mathbf{c}, \mathbf{z}$

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

(all operations are mod q)

What about hardness ?

Short integer solution (SIS)

Keygen() → sk, vk

- $\mathsf{vk} = \mathbf{A} \cdot \mathsf{sk}$, for short sk

Short integer solution (SIS)

Hint-LWE

Sign

- Sample a short $\mathbf{r}$
- $\mathbf{w} = A \cdot \mathbf{r}$
- $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w})$
- $\mathbf{z} = \mathbf{c} \cdot \mathsf{sk} + \mathbf{r}$
- Output $\mathbf{c}, \mathbf{z}$

Short integer solution (SIS)

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - c \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

(all operations are mod q)

Wait, MLDSA is that *simple* ?

Well... not really ! Actual MLDSA use smaller noise but
requires some probabilistic rejection on **z**
before outputting the final signature

Wait, MLDSA is that *simple* ?

Well... not really ! Actual MLDSA use smaller noise but
requires some probabilistic rejection on **z**
before outputting the final signature

let's forget about it ... for now

SIGNATURE SCHEME

+

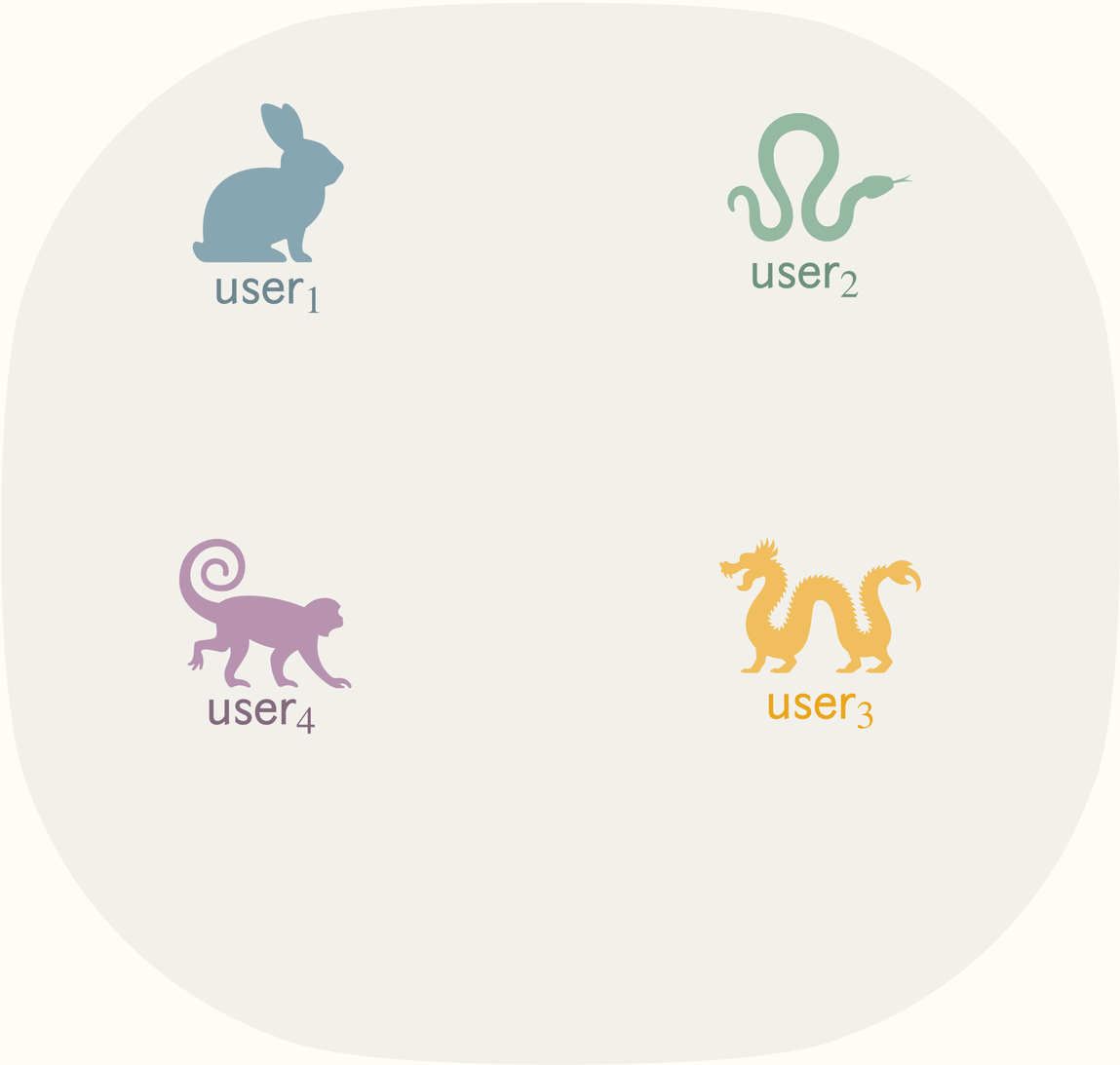
KEY DISTRIBUTION / SHARING

=

THRESHOLD SIGNATURE

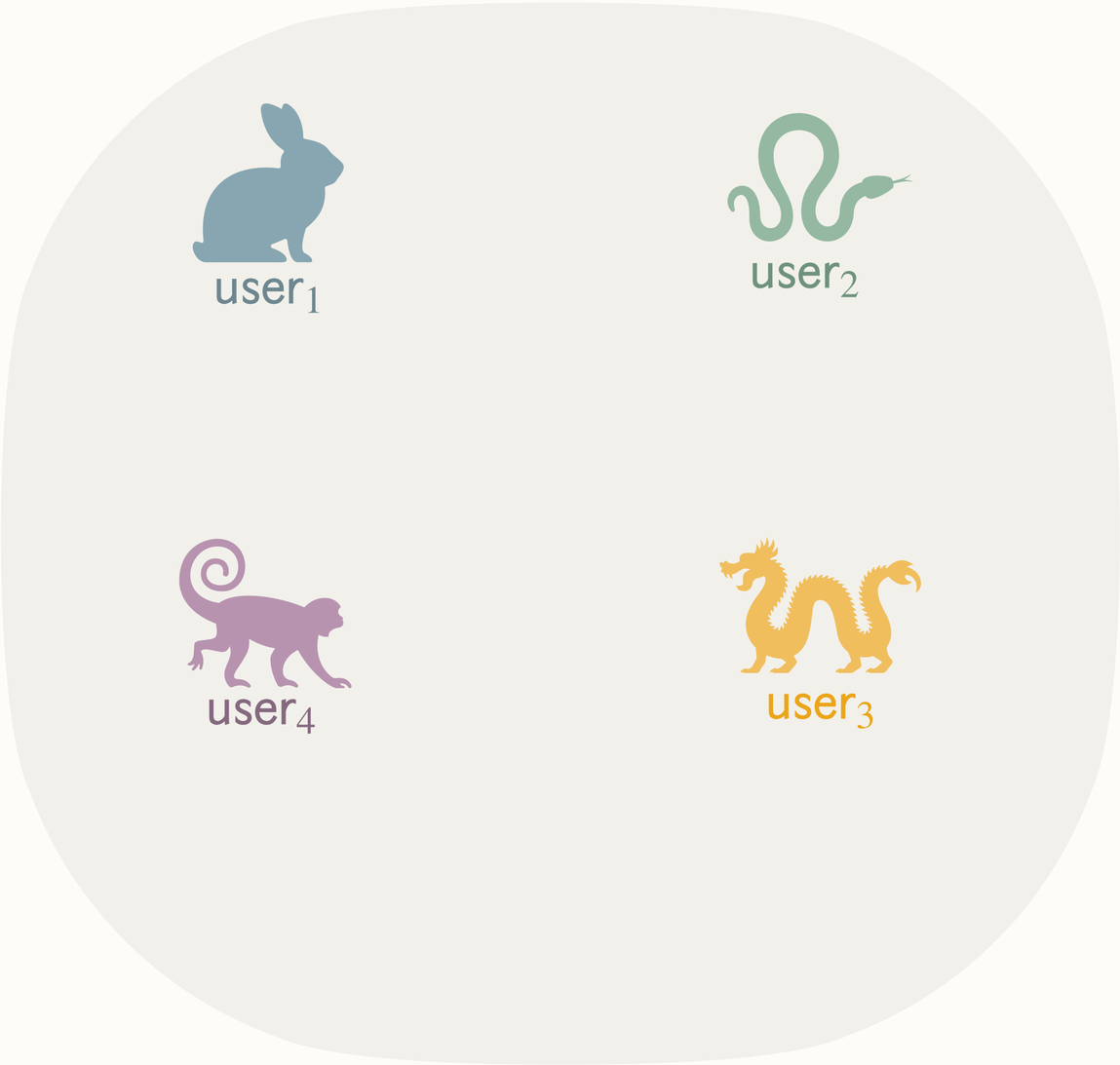
How to *shortly* share a secret?

secret sharing using *replication*



$(T, N) = (3,4)$

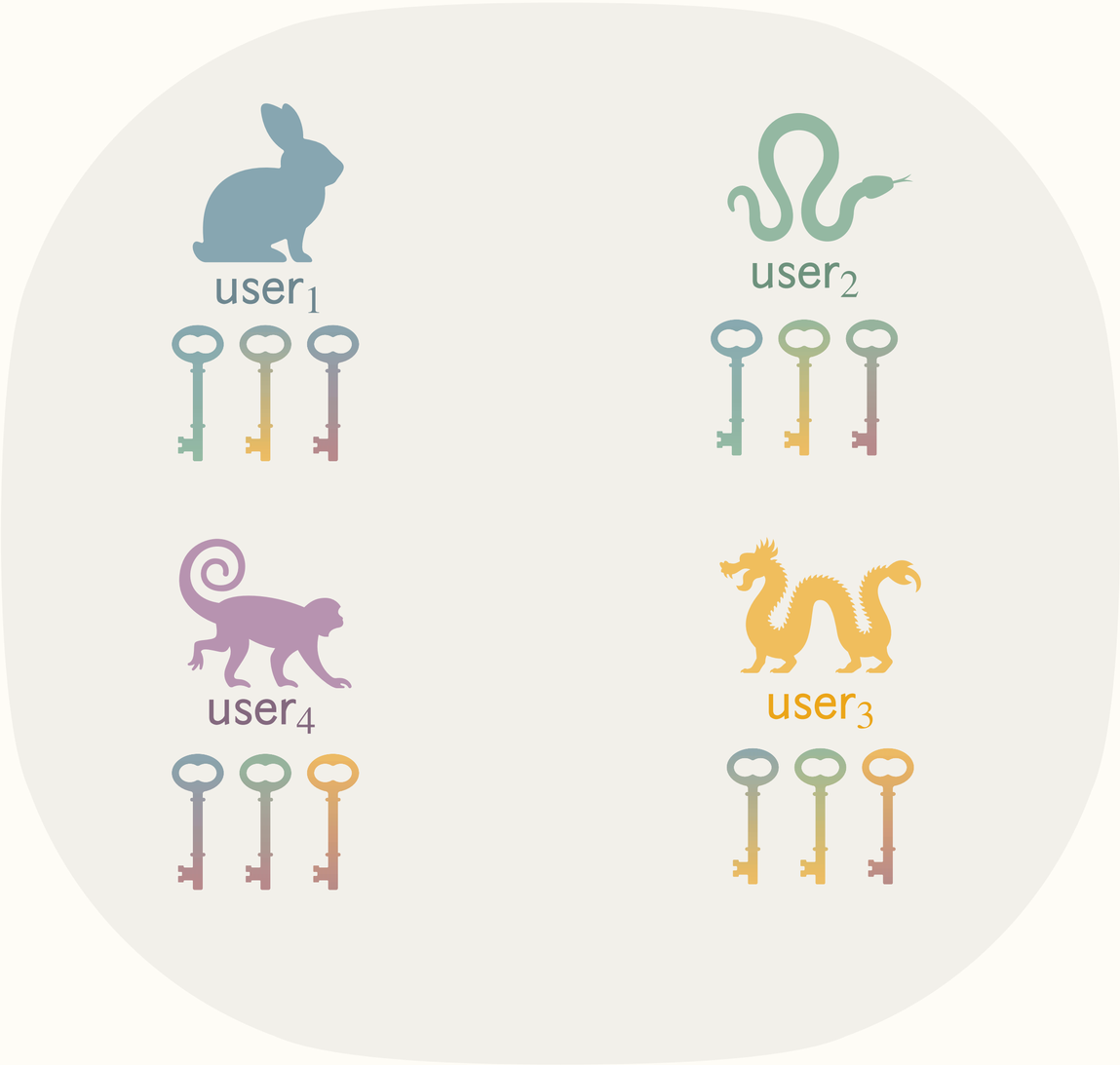
secret sharing using *replication*



$$(T, N) = (3,4)$$

$$sk = sk_{12} + sk_{13} + sk_{14} + sk_{23} + sk_{24} + sk_{34}$$

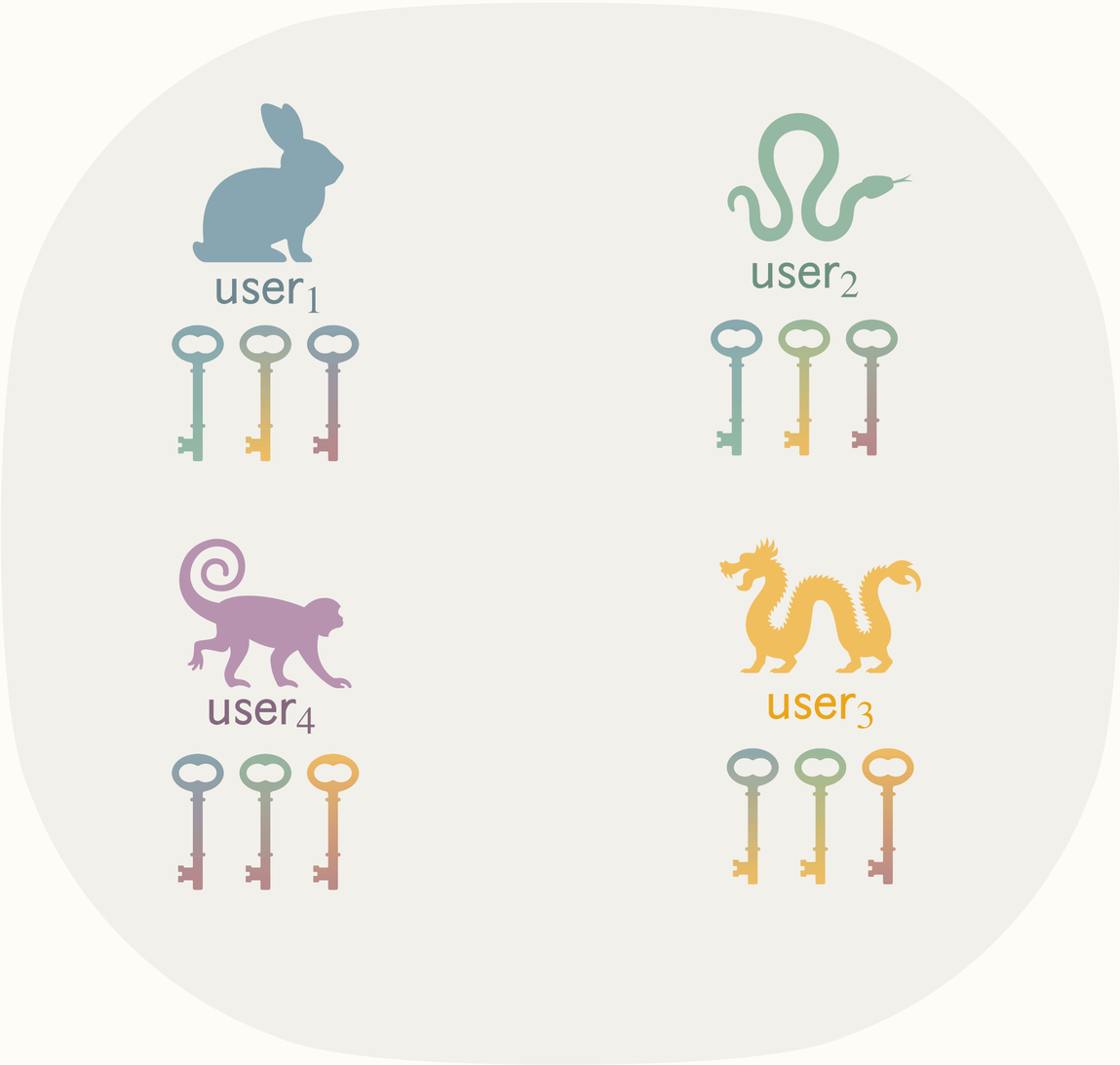
secret sharing using *replication*



$$(T, N) = (3,4)$$

$$sk = sk_{12} + sk_{13} + sk_{14} + sk_{23} + sk_{24} + sk_{34}$$

secret sharing using *replication*

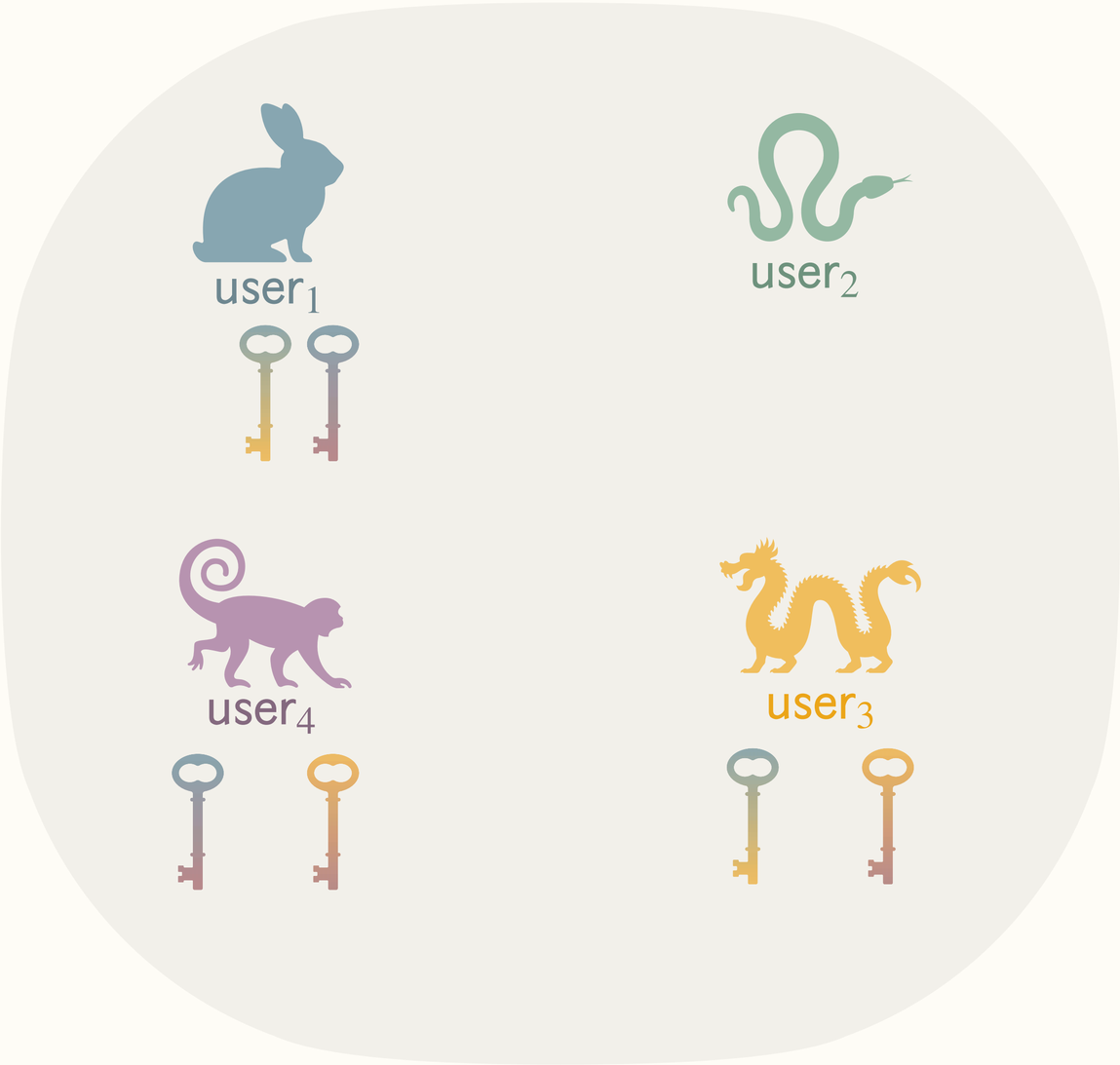


$$(T, N) = (3,4)$$

$$sk = sk_{12} + sk_{13} + sk_{14} + sk_{23} + sk_{24} + sk_{34}$$

$$I = \{1, 3, 4\}$$

secret sharing using *replication*



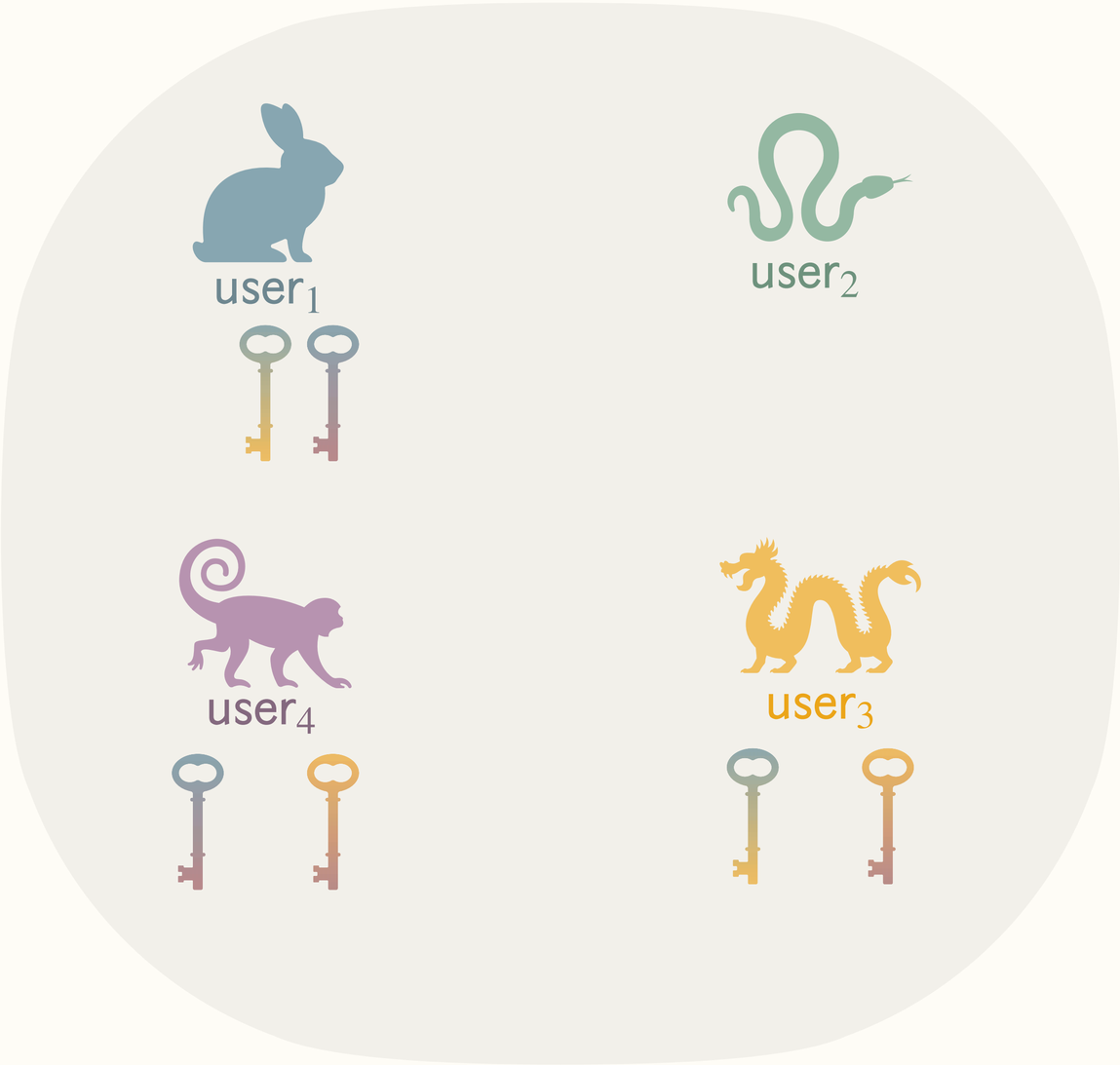
$$(T, N) = (3,4)$$

$$sk = sk_{12} + sk_{13} + sk_{14} + sk_{23} + sk_{24} + sk_{34}$$

$$I = \{1, 3, 4\}$$

$$sk = (sk_{12} + sk_{13}) + (sk_{23} + sk_{34}) + (sk_{14} + sk_{24})$$

secret sharing using *replication*

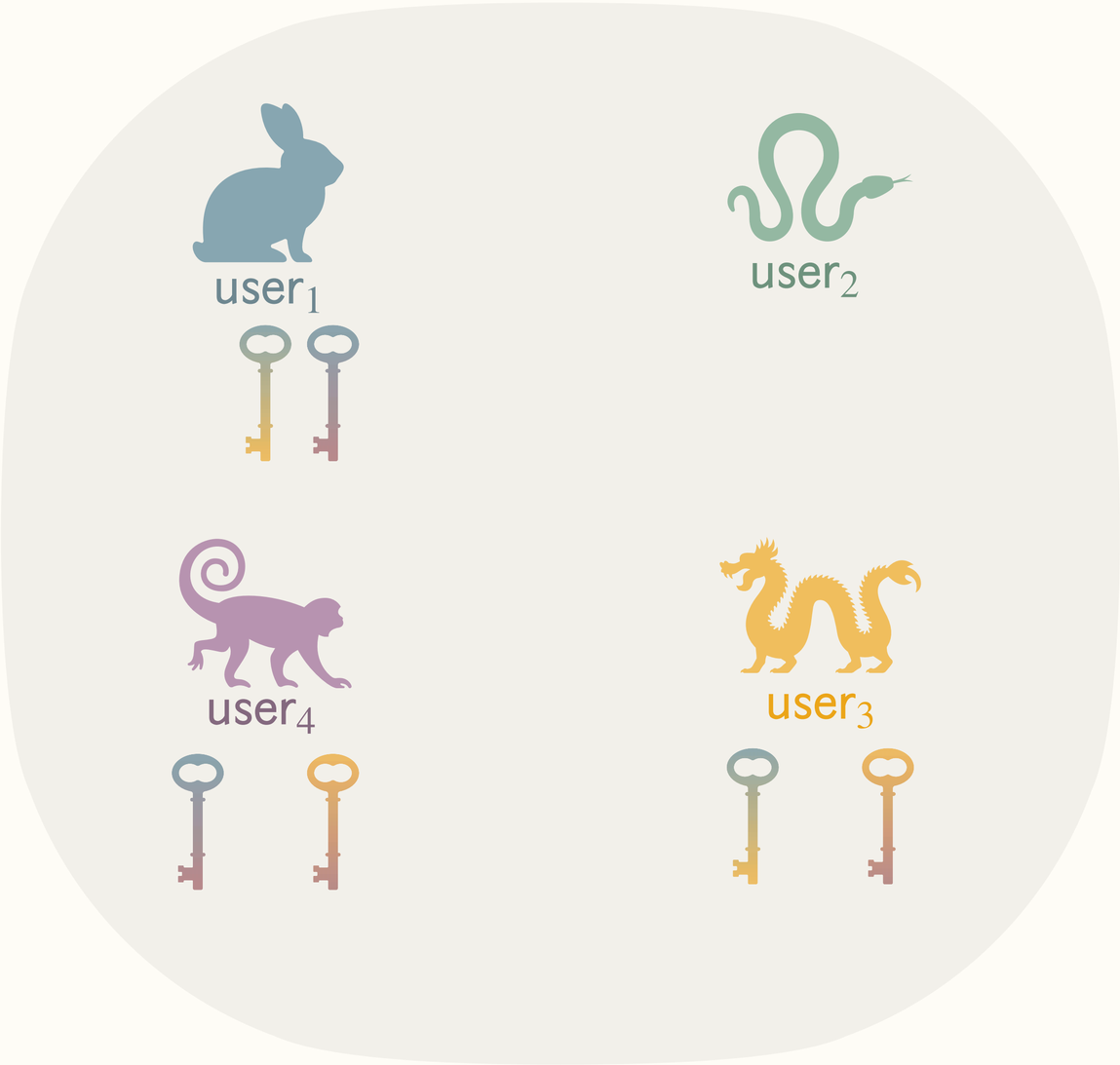


$$(T, N) = (3,4)$$

$$sk = sk_{12} + sk_{13} + sk_{14} + sk_{23} + sk_{24} + sk_{34}$$

$$I = \{1, 3, 4\}$$

secret sharing using *replication*



$$(T, N) = (3,4)$$

$$sk = sk_{12} + sk_{13} + sk_{14} + sk_{23} + sk_{24} + sk_{34}$$

$$I = \{1, 3, 4\}$$

$$sk = (sk_{12} + sk_{13}) + (sk_{23} + sk_{34}) + (sk_{14} + sk_{24})$$

Small shares/coefficients **but** requires *exponentially* many shares

SIGNATURE SCHEME

+

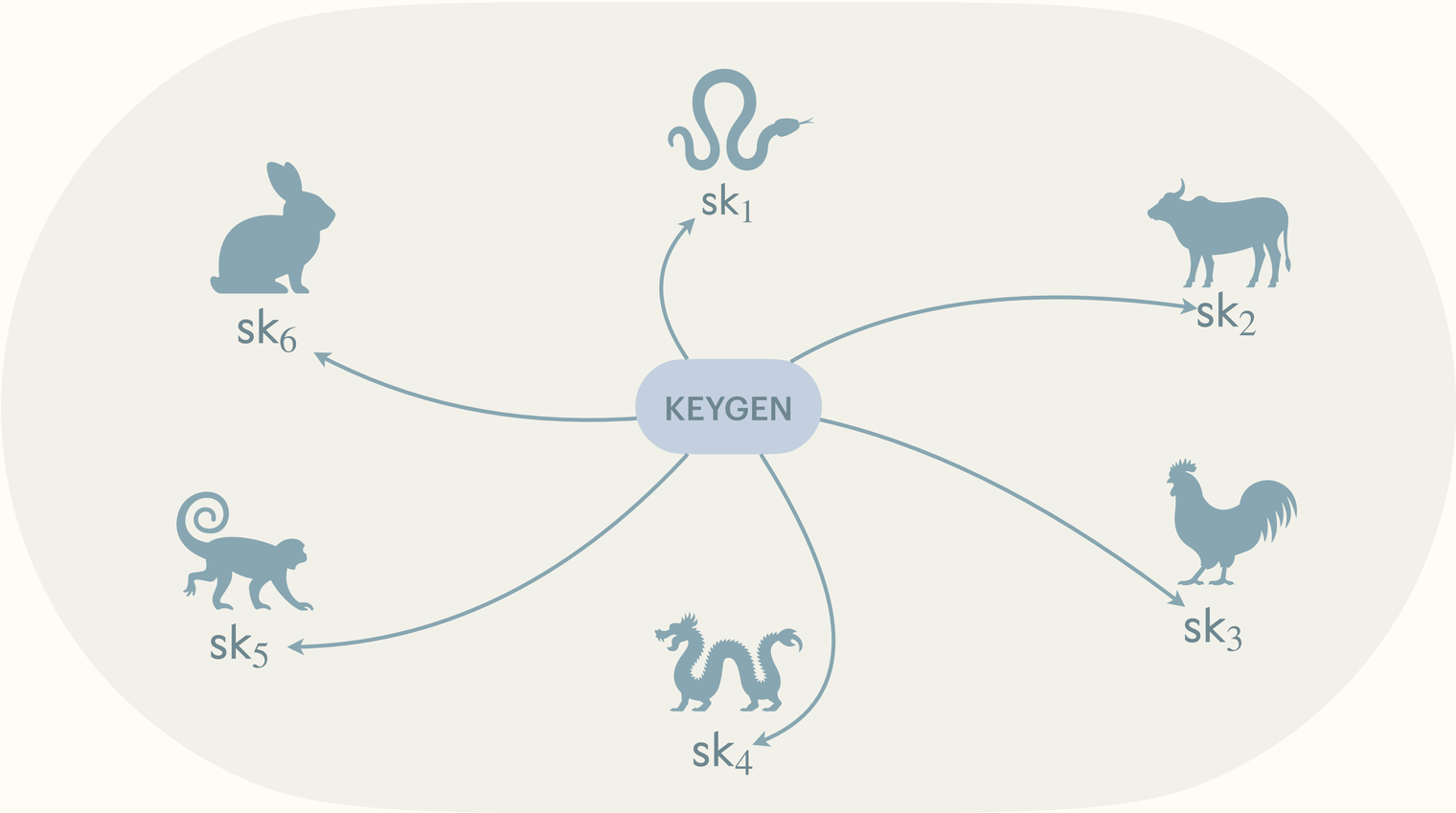
KEY DISTRIBUTION / SHARING

=

THRESHOLD SIGNATURE

BACK TO THE THRESHOLD

$$sk = sk_1 + sk_2 + sk_5$$



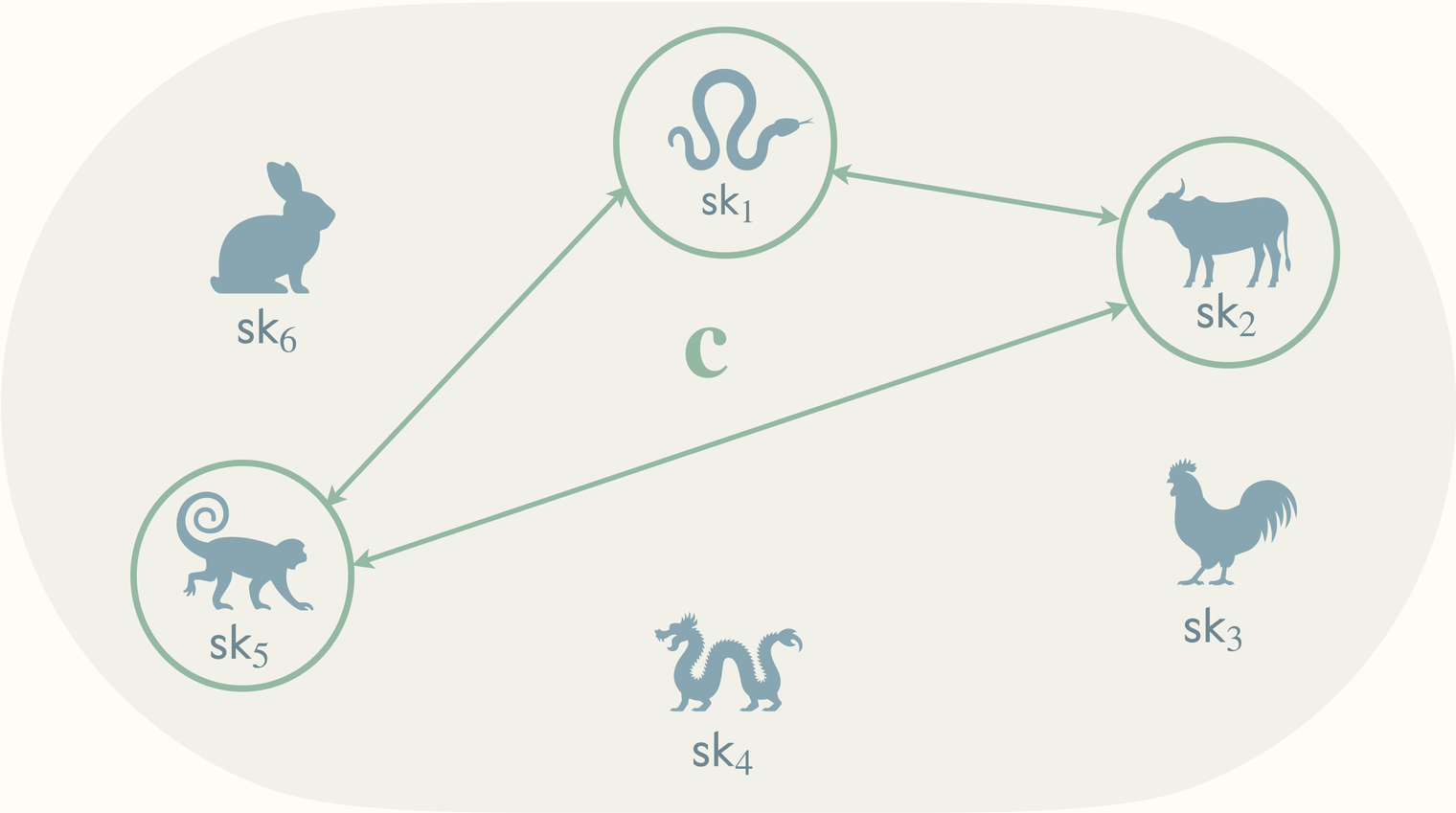
$$\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 + \mathsf{sk}_5$$

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - \mathbf{c} \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

1 - Agree on challenge $\mathbf{c}$

$$\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 + \mathsf{sk}_5$$



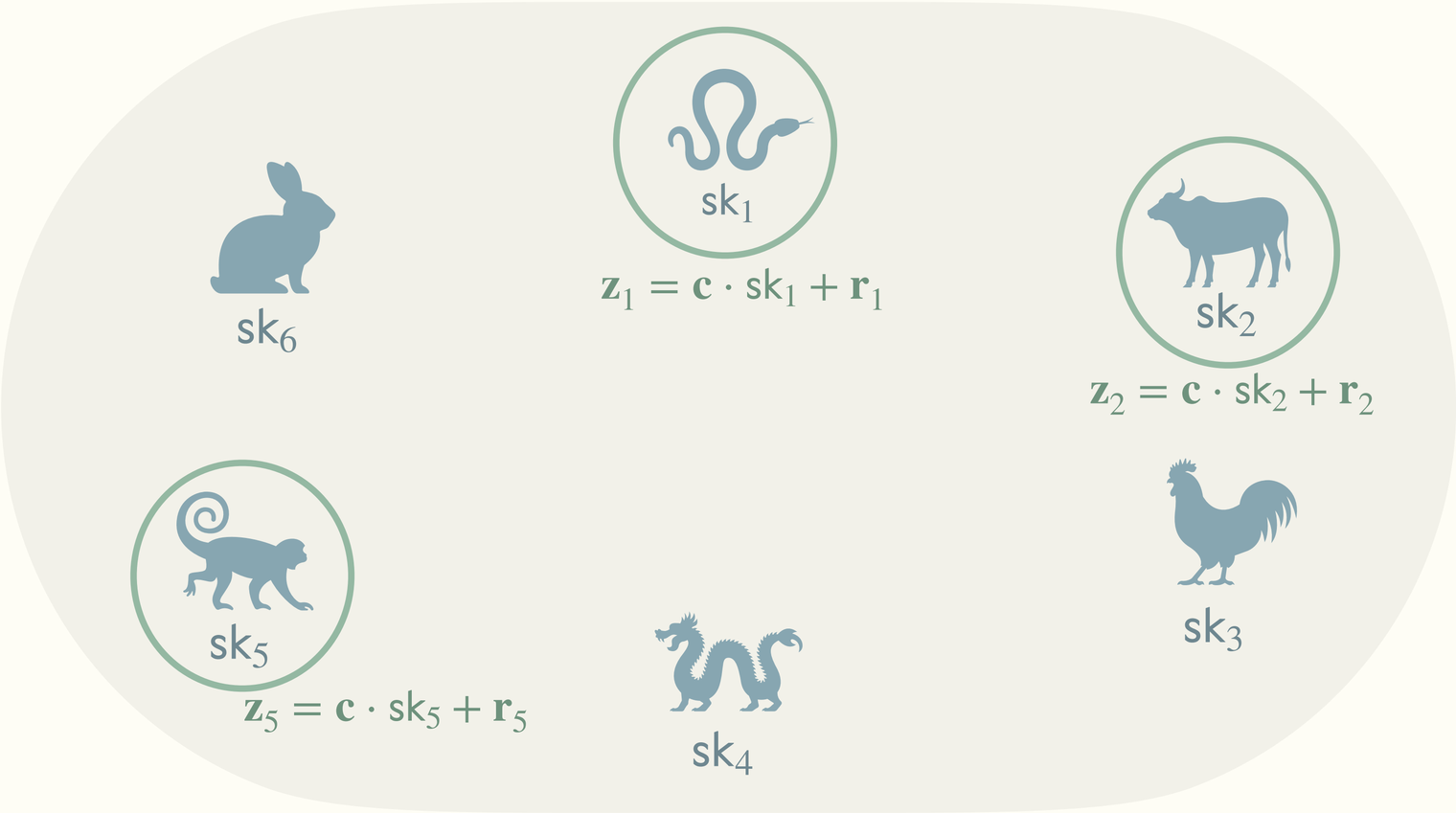
Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - \mathbf{c} \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

1 - Agree on challenge $\mathbf{c}$

2 - Compute the partial signature

$$\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 + \mathsf{sk}_5$$



Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - \mathbf{c} \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

1 - Agree on challenge $\mathbf{c}$

2 - Compute the partial signature

3 -Combine

$$\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 + \mathsf{sk}_5$$

$$\mathbf{z}_5 = \mathbf{c} \cdot \mathsf{sk}_5 + \mathbf{r}_5$$
$$\mathbf{z}_2 = \mathbf{c} \cdot \mathsf{sk}_2 + \mathbf{r}_2$$
$$\mathbf{z}_1 = \mathbf{c} \cdot \mathsf{sk}_1 + \mathbf{r}_1$$

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - \mathbf{c} \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

1 - Agree on challenge $\mathbf{c}$

2 - Compute the partial signature

3 -Combine

$$\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 + \mathsf{sk}_5$$

$$\begin{aligned} \mathbf{z}_5 &= \mathbf{c} \cdot \mathsf{sk}_5 + \mathbf{r}_5 \\ \mathbf{z}_2 &= \mathbf{c} \cdot \mathsf{sk}_2 + \mathbf{r}_2 \\ + \quad \mathbf{z}_1 &= \mathbf{c} \cdot \mathsf{sk}_1 + \mathbf{r}_1 \end{aligned}$$

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - \mathbf{c} \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

1 - Agree on challenge $\mathbf{c}$

2 - Compute the partial signature

3 -Combine

$$\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 + \mathsf{sk}_5$$

$$\begin{aligned} \mathbf{z}_5 &= \mathbf{c} \cdot \mathsf{sk}_5 + \mathbf{r}_5 \\ \mathbf{z}_2 &= \mathbf{c} \cdot \mathsf{sk}_2 + \mathbf{r}_2 \\ + \quad \mathbf{z}_1 &= \mathbf{c} \cdot \mathsf{sk}_1 + \mathbf{r}_1 \end{aligned}$$

Combine

Output $(\mathbf{c}, \mathbf{z}_1 + \mathbf{z}_2 + \mathbf{z}_5)$

Verify

- $\mathbf{w}^* = A \cdot \mathbf{z} - \mathbf{c} \cdot \mathsf{vk}$
- Assert $\mathbf{c} = \mathsf{Hash}(m, \mathbf{w}^*)$
- Assert $\mathbf{z}$ short

Wait, th-MLDSA is that *simple* ?

let's not forget about it, now.

Well... not really ! Actual Th-MLDSA
requires some probabilistic rejection
before outputting the final signature

What does this mean

in practice ?

Let's look first at practical ideas

What do we need?

- **PRACTICALITY** create a signature with high probability
- **COMPATIBILITY** signatures should be valid for FIPS-204 ML-DSA verification

  (*But! We are not indistinguishable from the original MLDSA*)
- **SECURITY** Secure under real-world assumptions:
  - ✓ Dishonest-majority
  - ✓ Active security
  - ✓ Arguably adaptive security

The crux: *double rejection sampling*

- Remember!
  - **Signatures are aborted and retried (abort-and-retry paradigm)**
  - **Rejected candidates should not be leaked**
  - **ML-DSA uses infinity norm (rejection region = hypercube)**

- How can we do this?

1. Per-party rejection of partial signatures (security) [on partial $\mathbf{z_i}$]
2. Global rejection at the time of aggregation (compatibility) [on final $\mathbf{z}$]

Beyond : back to *rejection sampling*

Signatures are too big for passing *MLDSA's verification*

→

Add rejection

motto : “reject until the distribution of signatures is small enough ”

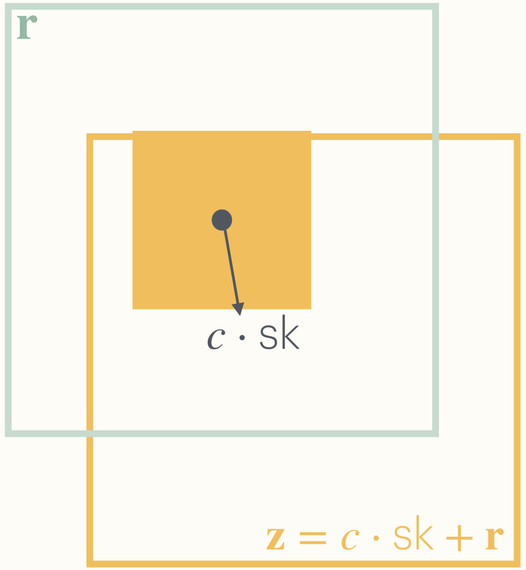
<u>no free lunch:</u> the further away my starting distribution is, the more reject I have to do.

Enter: Optimised *rejection sampling*

- Parties perform rejection sampling in parallel
- If all *T* parties perform this independently:
  - Single-party probability *p*
  - Exponential: threshold acceptance probability *p^T*
  - Exponential degration

Exploit slack on ML-DSA norm bounds:

- Single-party ML-DSA:
  - Sample in uniform hypercube
  - Reject if outside the bound
  - But!
    - Verification region is looser than necessary
    - There is an unused geometric slack



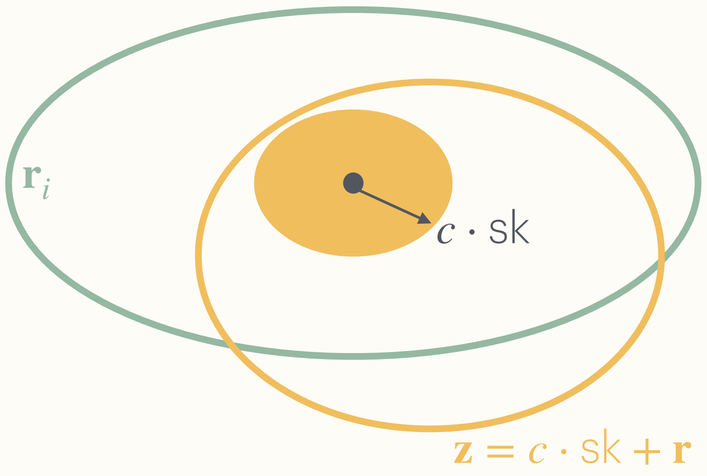
Enter: Optimised *rejection sampling*

Exploit slack on ML-DSA norm bounds:

- Single-party ML-DSA:
  - Sample in uniform hypercube
  - Reject if outside the bound
  - But!
    - Verification region is looser than necessary
    - There is an unused geometric slack

Threshold ML-DSA

- Sample from a tighter region: hyperball
- Reject outside
- Run *K* independent attempts until success



Let's look first at practical ideas

What do we need?

- **PRACTICALITY** create a signature with high probability
- **COMPATIBILITY** signatures should be valid for FIPS-204 ML-DSA verification
- **SECURITY** Secure under real-world assumptions

→

- Compact signature size
  - Limited # of parties
- Not indistinguishable: we do not sample over uniform as in ML-DSA
- Unforgeability holds under the same ML-DSA assumptions

Beyond : back to *rejection sampling*

Signatures are too big for passing *MLDSA's verification*

Add rejection

Needs to carefully control the parameters of the rejection

- ✓ **Works !**
- ✗ **Can’t scale beyond 6 users with the current technology**

What do we get ?

- **EFFICIENCY** 2 offline and 4 online rounds (which is the state-of-the-art)
- **PRACTICALITY**
  - The first round is independent of the message
  - We can pre-process so that online times remain minimal
  - Allows for safe concurrency

Let's look at numbers

Communication costs for Th-MLDSA at *N* parties with threshold *T* for a success probability of 1/2

N\T	2	3	4	5	6
2	10.5 kB				
3	15.8 kB	21.0 kB			
4	15.8 kB	36.8 kB	42.0 kB		
5	15.8 kB	73.5 kB	157.4 kB	84.0 kB	
6	21.0 kB	99.8 kB	388.4 kB	524.8 kB	194.2 kB

Local Timings for Th-MLDSA on a MacBook M3 (24 GB of RAM)

(T, N)	KeyGen (ms)	Sign+Combine (ms)	Verify (ms)
Threshold ML-DSA			
(3,3)	0.3669	0.6810	
(2,4)	0.1709	0.4570	
(3,4)	0.2062	1.0961	
(4,4)	0.1655	1.3672	
(3,5)	0.2870	2.2263	0.0306
(4,5)	0.2940	4.9832	
(5,5)	0.1956	2.8453	
(4,6)	0.5016	12.1949	
(6,6)	0.2181	7.6784	

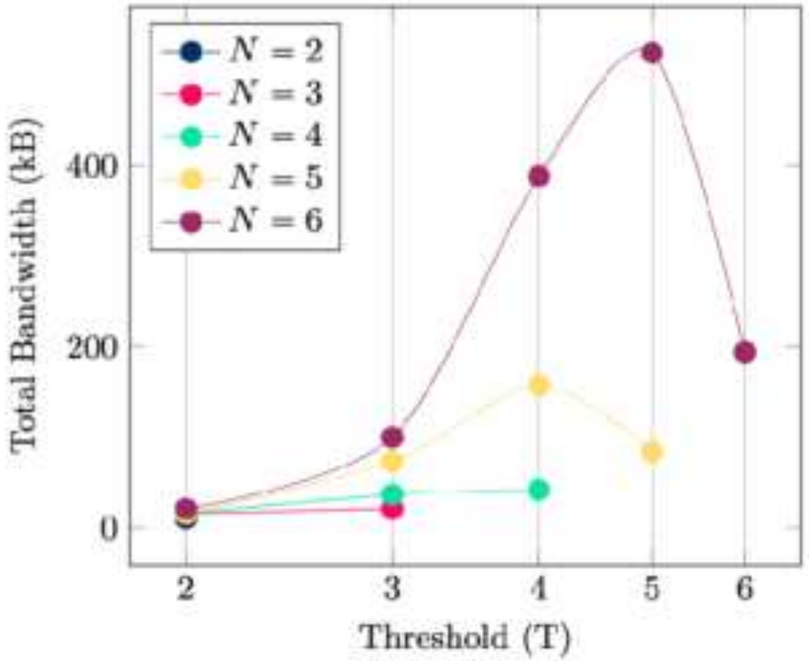
Implementation in `Golang` (https://github.com/Threshold-ML-DSA/Threshold-ML-DSA):

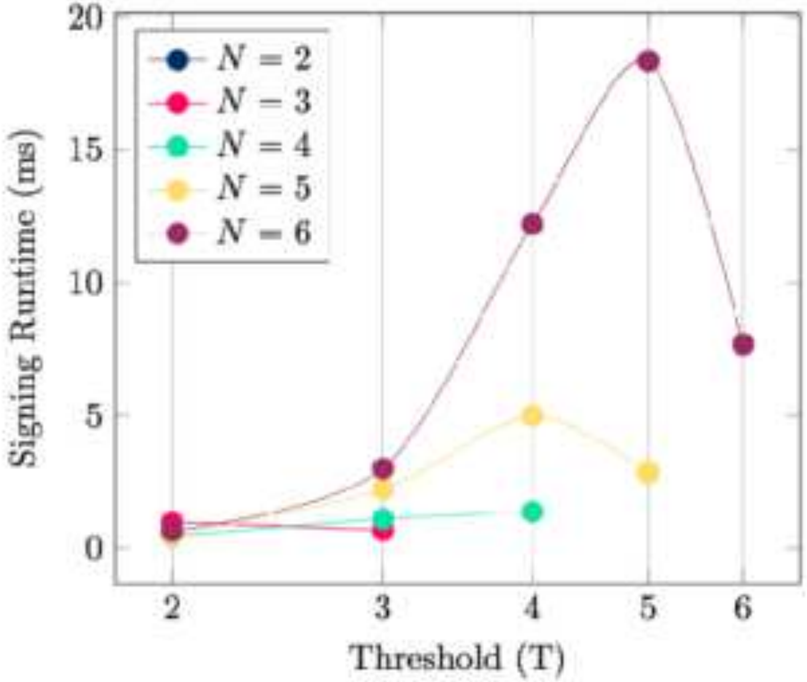
- ➡ We did not yet optimise neither at the software or hardware level
- ➡ Support all ML-DSA security levels

And numbers?

Parameters aim for a success probability 1/2 for each attempt (vs ~1/4 in original ML-DSA).

Efficient up to 6 parties.





Bandwidth and latency of threshold signing for ML-DSA 44 (on a local network)

Parties are executed in parallel, and we average over successful attempts.

Let's look at numbers

Table 1: Comparison of approaches to thresholdize ML-DSA-44, with per party metrics averaged over successful signing attempts.

Scheme	# Parties (upper bound)	# Rounds Off + Online	Communication Off + Online (MB)	Computation	Model	Security
This work	6	2+4	$10^{-5} + 0.021$ to 1.05	Lightweight	Game-based	Dishonest majority
Bienstock et al. [7]	Unlimited	$136^{*} + 79$ $37^{*} + 23$	$> 0.87^{*} + 0.70$ $> 1.9^{*} + 1.5$	Online lightweight*	UC	Honest majority
Trilithium [29]	2	$1^{\dagger}$+60	$234^{\dagger} + 0.40$	Heavy	UC	Trusted party†
Generic MPC [17]	Unlimited	High	High	Impractical	UC	Dishonest majority

[7] Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Efficient, scalable threshold ML-DSA signatures: An MPC approach. Cryptology ePrint Archive, Paper 2025/1163, 2025.

[29] Antonín Dufka, Semjon Kravtšenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ML-DSA signing. Cryptology ePrint Archive, Paper 2025/675, 2025.

[17] Daniele Cozzo and Nigel P. Smart. Sharing the LUOV: Threshold post-quantum signatures. 17th IMA International Conference on Cryptography and Coding, volume 11929 of LNCS, pages 128– 153. Springer, Cham, December 2019.

Lower communication and computational cost than state-of-the-art

What about WAN?

Requirement: emulate a *real-world experiment*:

- We implemented our scheme inside *libp2p* (https://libp2p.io/), a modular peer-to-peer network stack, that works over TCP streams
  - A mesh network topology, where messages are sent in parallel

Results

- Our results show that the overall impact is the critical path (slowest communication path) due to latency
- All costs are well under a second

What about WAN?

T = 2,

N = 6,

Taipei ⟷ Seoul

Th-ML-DSA-44

What about WAN?

T = 2,

N = 6,

Taipei ⟷ Virginia

Th-ML-DSA-44

What about WAN?

T = 2,

N = 6,

Taipei ⟷ Seoul

⟷ London

Th-ML-DSA-44

Can we use it?

Post-quantum PKI and TLS infrastructure

We are well under the TLS timeout of 10-20 seconds

Cryptocurrency wallets

Users prefer high reputable parties and low threshold values

Critical infrastructure (Tor Network)

Small number of directory authorities that need to generate randomness

Our two biggest use cases for a *more private* Internet

Future work

- Optimised implementation in a robust language
- Practical evaluation on small number of parties
- Can we make signatures indistinguishable while still being efficient?
- Interested in practical applications that preserve privacy of the Internet

Submission to the NIST call for threshold schemes

https://mithril-th.org/

Thank you